

WEEK 7 PROGRESS UPDATE

Multilingual Speech-to-Intent Detection System

Research Progress Report

Prepared by	Y. Unmesha Kruthika	Date	September 11, 2026
Reporting Period	Week 7	Status	Ongoing — Synthetic Audio Dataset Built; Fine-Tuning Not Yet Started

1. Purpose of This Report

The last two reports (Weeks 5 and 6) worked from a two-stage view of the pipeline: an ASR model converts speech to text, and a separate text SLM classifies that text into one of the 60 MASSIVE intents. This week I started exploring a different, single-model direction instead — an architecture where a speech model takes audio directly as input and predicts the intent directly as output, with no separate transcription step in between. This report documents what I have actually completed so far on that direction: building a synthetic audio training set, since no public Indian-language audio intent dataset exists. Fine-tuning a model on that dataset has not started yet — the experiment is still in progress, and this report is a checkpoint on where it stands.

2. Why a Direct Audio-to-Intent Model

Speech → Audio Model → Intent (60 classes)

Instead of chaining an ASR model and a text classifier, this direction fine-tunes one speech model to read an audio waveform and output an intent label directly. The appeal is a simpler pipeline with one point of failure instead of two, and no dependency on transcription quality — a wrong ASR transcript can no longer drag down an otherwise-correct intent decision. The trade-off is that a model trained this way needs an audio-labelled dataset with intent tags attached to actual speech, not text.

3. The Missing Piece: No Public Indian-Language Audio Intent Dataset

MASSIVE, the dataset used throughout this project, is text-only — every one of its 60 intents across the five Indian locales (Hindi, Kannada, Malayalam, Tamil, Telugu) exists only as written utterances, never as recorded speech. Searching for a ready-made Indian-language audio intent dataset did not turn up anything public that maps directly onto this project's intent taxonomy. Rather than wait on that, the approach this week was to synthesise the audio side myself: take MASSIVE's existing text utterances and generate speech for each one with a text-to-speech (TTS) model, so that every audio clip inherits the correct intent label directly from the MASSIVE record it was generated from.

4. Generating the Synthetic Audio Dataset

4.1 The TTS model used: Facebook MMS-TTS

The model used to generate audio is Facebook's MMS-TTS, the text-to-speech component of Meta's Massively Multilingual Speech (MMS) project. It is a VITS-style model — VITS trains an end-to-end neural text-to-speech system that combines a variational autoencoder with adversarial training, so a single model goes directly from text to a raw waveform rather than needing a separate vocoder stage. It was chosen for three practical reasons: it ships inside the transformers library already, so it needs no extra packages beyond what the project already installs; it has a released checkpoint for every one of the five target languages; and it outputs 16 kHz audio, which is the input rate that speech models in this project's family (Whisper, wav2vec2-based encoders) expect, so no resampling step is needed later. The exact checkpoints used were facebook/mms-tts-hin, facebook/mms-tts-kan, facebook/mms-tts-mal, facebook/mms-tts-tam and facebook/mms-tts-tel — one dedicated checkpoint per language, loaded through Hugging Face's VitsModel and AutoTokenizer classes.

4.2 Building the dataset

- Sampled a stratified subset of MASSIVE covering all 60 intents (not a 5-intent smoke test), so every class is represented in the audio dataset, with the number of clips per intent per language configurable and started small before scaling up.
- Generated one .wav clip per sampled utterance using the matching per-language MMS-TTS checkpoint, and wrote an incremental manifest CSV that maps every audio file back to its MASSIVE intent id — the file Stage 3 fine-tuning will actually consume.
- Made the generation loop resumable: a clip that already exists on disk is skipped and the manifest is flushed periodically, so a disconnected Colab session loses no completed work and can simply be re-run.
- Repeated the same TTS process separately for MASSIVE's official validation and test splits, with two additional safeguards — skipping empty or single-character utterances that had earlier caused generation errors, and discarding any output clip shorter than 0.2 seconds instead of keeping a broken file.

5. Dataset Produced So Far

Split	Source	Approx. Clips	Coverage
Train	Stratified sample of MASSIVE train, all 5 languages	~39,000	All 60 intents
Validation	Full official MASSIVE validation split, all 5 languages	included in the combined ~25,000 val + test clips	All 60 intents
Test	Full official MASSIVE test split, all 5 languages	included in the combined ~25,000 val + test clips	All 60 intents

Every clip is 16 kHz mono audio synthesised from a real MASSIVE utterance, so it carries the same intent label as the text record it came from, across all five languages and all 60 intent classes.

6. Verifying the Dataset Before Using It

- Checked per-language clip-duration statistics (count, mean, min, max) and flagged suspiciously short clips (under 0.3 seconds) as likely failed generations rather than assuming every file is valid.
- Listened to one sample per language from the generated manifest to confirm the audio is not silent, not clipped, and is actually in the intended language before scaling generation up further.
- Confirmed there is zero overlap in utterance ids between the train, validation and test manifests, checked pairwise across all three splits — important because MASSIVE ids are shared across languages, so this had to be verified globally rather than per language.

With the dataset built and verified, the audio side of this direction is in place. The step not yet done is fine-tuning an actual model on it — a draft training notebook exists for reference, but it has not been run, so no accuracy numbers exist yet for the audio-in, intent-out direction.

7. Candidate Models for Direct Audio-to-Intent Fine-Tuning

Since the model has not been fine-tuned yet, this section lists the candidates under consideration for that step, rather than reporting results.

Model	Approx. Size	Pretraining Coverage	Note
Wav2Vec2 XLS-R (facebook/wav2vec2-xls-r-300m)	300M	128 languages, incl. Hindi, Tamil, Telugu, Kannada, Malayalam	Current default in the draft notebook; fits a free-tier T4 GPU with fp16 and gradient checkpointing
MMS speech encoder (facebook/mms-300m)	300M	Same MMS project as the TTS model already used, 1000+ languages	Drop-in alternative to XLS-R noted in the draft notebook; same architecture, different pretraining data
Whisper (e.g. openai/whisper-small) as a classifier	~240M (small)	99 languages, strong reported performance on Indic speech	Used via WhisperForAudioClassification instead of its normal decoder; heavier than XLS-R
Wav2Vec2 XLS-R (facebook/wav2vec2-xls-r-1b)	1B	Same 128-language coverage as the 300M version, larger capacity	Noted as an option but needs an A100-class GPU rather than a free-tier T4
AI4Bharat IndicWav2Vec2	~95M–300M (base/large)	Self-supervised pretraining focused specifically on Indian languages	Not yet tried in the notebook; worth benchmarking given its India-specific pretraining

The draft training notebook currently targets XLS-R 300M specifically: audio goes through the wav2vec2 encoder, is mean-pooled, and passed through a linear head over the 60 intent classes — the same recipe used for intent classification in the SUPERB benchmark. It also freezes the model's CNN feature extractor and enables weighted layer summation, so the classifier can learn which transformer layer carries the most useful signal for intent rather than only reading the final layer, plus SpecAugment-style

masking and audio augmentation (speed perturbation, gain, noise) applied during training. None of this has been executed yet — it is prepared and ready to run once time is available for the fine-tuning step.

8. Notes and Constraints

- Fine-tuning has not been run. The training notebook referenced this week is a prepared pipeline, not a completed experiment — there are no accuracy or F1 numbers yet for the audio-in, intent-out direction.
- The entire training set is synthetic, single-voice-per-language TTS audio. It will not capture the variation of real speakers, accents or recording conditions, so strong performance on this synthetic test set would not by itself confirm the model works on real recorded speech.
- MMS-TTS was chosen for convenience and language coverage, not because it was benchmarked against other TTS options; a second TTS system could be introduced later to add voice diversity to the training set if the synthetic-to-real gap turns out to be large.
- The candidate model list in Section 7 is a starting shortlist based on parameter size and language coverage as published by each model's developer — none of them have been evaluated on this project's synthetic dataset yet.